

上海人工智能实验室 · 联合学术界与产业界共同编著

# 超节点技术体系白皮书

SuperPod Technical White Paper

v1.0 · 2026 年 3 月发布 · 联合 8 所高校及科研机构、16 家产业伙伴共同编著

超节点是一种极致的系统工程路径，通过不断引入新的技术变量来实现超越摩尔定律的算力增长，进而满足指数增长的算力需求。

## 全书结构

从需求压力出发，理解超节点为何出现、系统能力如何兑现，再到工程路径选择与未来演进

为什么需要超节点，以及怎么把系统能力兑现出来

### 第一章

## 架构分析

算力需求四波叠加，单芯片已接不住系统增速。从需求与供给的张力出发，定义超节点并建立分析框架。

### 第二章

## 软件系统

硬件打开能力上界，软件决定能兑现多少。讨论内存语义、通信运行时与 RAS 如何把硬件能力转化为有效算力。

### 第三章

## 建模仿真

判断不能停在规格表比较。建模仿真同时回答：理论上界在哪、实际兑现了多少、哪些新变量正在推动系统继续进步。

## 当前怎么选工程路径，未来哪些技术将改变格局

### 第四章

## 参考设计

五个参考设计分别面向不同约束和负载，代表当前五种各有优劣、互不替代的工程路径。

### 第五章

## 未来演进

光互联、先进封装、Chiplet、HBM/3D DRAM 与模型形态演进，正在改写下一代系统的技术格局。

第六章

总结

真正决定竞争格局的，不是某个局部参数，而是持续提升系统整体能力的速度。

从分析框架到产业协作

SPI

SuperPod Pareto Index

白皮书回答“怎么看”，SPI 则讨论如何把比较框架落地：产品条目怎么定、数据怎么对齐、边界怎么划，为后续持续评估奠定基础。

产业生态

产业生态地图

超节点是跨芯片、互联、封装、软件与运营的协同工程。产业生态地图识别关键环节、短板与优先投入方向。

牵头单位

学术合编单位

产业合编单位

# 架构分析

前言已经提出了一个判断：下一代算力建设的关键，不再只是采购更强芯片或部署更多节点，而是能否把计算、互联、内存、软件与整机组织成一种可持续演进的系统能力【研判】。顺着这一判断继续往下追问，第一章真正要面对的，其实不是“超节点长什么样”，而是一个更根本的问题：在制程红利放缓之后，什么样的系统对象还能持续把系统算力向外推？如果这个问题没有被讲清，超节点就很容易被误解为“把更多卡装进一个机柜”的工程放大版。更准确地说，超节点不是形态概念，而是局部性边界被重新定义后的系统对象：它试图把更多高价值通信、远端访存、同步协同和恢复控制稳定收敛在同一个受控域内，这个域既要低时延、低抖动，也要具备可维持内存语义或近内存语义、故障域可控并最终兑现 Goodput 的能力。

本章试图给出的解释是：压力首先来自需求侧，而突破发生在供给侧【归纳】。需求侧并非单一增长曲线在推动算力基础设施演进，而是至少有两股力量同时施压【归纳】。一方面，大模型的参数-性能幂律仍在持续推高训练与推理所需计算量；另一方面，AI for Science 与复杂工程问题的求解又受到边际递减约束，迫使系统不断堆高可用算力。对应地，供给侧实际观测到的约每两年 5–6 倍系统算力增长，已经远超制程微缩所能解释的范围【验证】。更合理的解释是：每一代架构师都在封装、互联、内存、精度、软件和整机工程等层面引入新的系统级设计变量，使原本不可同时达到的一组性能、成本与复杂度组合变得可达，并推动系统能力边界持续外移【归纳】。超节点之所以重要，正因为它把封装、互联、内存、软件和整机工程收束为同一个局部性边界的重构问题【归纳】。**超节点竞争的本质，也因此是系统能力边界外移速度之争【研判】。**

沿着这条线索，本章按从「为什么」到「怎么做」的顺序展开。本章先从需求侧的双重指数压力出发，说明系统能力边界为何必须被重新理解，再用帕累托前沿把这种边界形式化；在此基础上，进一步澄清这里讨论的 Scale-Up 并非狭义上的「卡间网络」或「更快互联」，而是一种在功耗、布线、散热与故障域约束下，**把更多高价值通信稳定留在受控域内的系统能力**。与之对应，Scale-Out 负责把节点或超节点继续横向扩展为更大集群，Scale-Across 则把这种资源组织延伸到跨园区、跨中心与跨资源域协同。最后，本章逐层展开**产品实践**（含 NVIDIA、华为 等）、**电气接口、互联协议、网络拓扑、系统架构与整机工程**，解释超节点究竟如何在机柜尺度上把这些变量组织成一个真正可交付的系统。

## 算力需求的指数增长

对算力基础设施的指数级需求，并非来自单一驱动力，而是至少有两条独立的增长曲线在同时施压。【归纳】

### 大模型的 Scaling Law

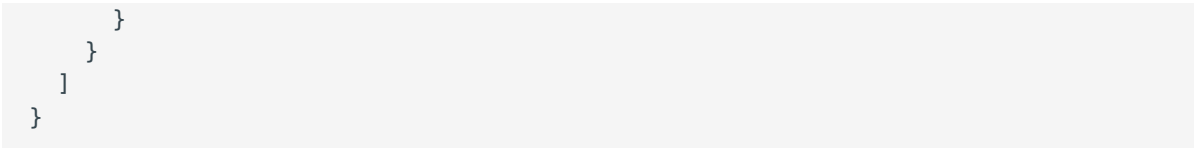
前沿生成式 AI 模型的训练算力需求呈现出稳定的指数增长。从 2020 年的 GPT-3（约  $\$3.14 \times 10^{23}$  FLOPs）到 2025 年的 Grok-3（约  $\$3.5 \times 10^{26}$  FLOPs），5 年间增长超过三个数量级。Kaplan et al. (2020) 与后续研究表明，模型性能（以损失函数衡量）与训练计算量之间存在幂律关系——要获得可感知的性能提升，所需计算量必须按固定比例增长[ $\text{scaling}$ ]。这条幂律没有显示出收敛迹象。

```
{
  "$schema": "https://vega.github.io/schema/vega-lite/v5.json",
  "description": "Training compute of frontier AI models over time.",
  "width": 700,
  "height": 500,
  "data": {
    "values": [
      {"model": "Meena", "organization": "Google", "date": "2020-01-28",
"compute": 1.12e+23, "parameters": 2600000000},
      {"model": "GPT-3 175B", "organization": "OpenAI", "date": "2020-05-28",
"compute": 3.14e+23, "parameters": 17500000000},
      {"model": "GShard", "organization": "Google", "date": "2020-06-30",
"compute": 4.77e+22, "parameters": 2300000000},
      {"model": "Switch", "organization": "Google", "date": "2021-01-11",
"compute": 8.22e+22, "parameters": 15710000000},
      {"model": "FLAN 137B", "organization": "Google", "date": "2021-09-03",
"compute": 2.05e+24, "parameters": 13700000000},
      {"model": "Gopher", "organization": "Google", "date": "2021-12-08",
"compute": 6.31e+23, "parameters": 2800000000},
      {"model": "PaLM", "organization": "Google", "date": "2022-04-04",
"compute": 2.53e+24, "parameters": 5400000000},
      {"model": "GPT-4", "organization": "OpenAI", "date": "2023-03-15",
"compute": 2.1e+25, "parameters": 1800000000},
      {"model": "PaLM 2", "organization": "Google", "date": "2023-05-10",
"compute": 7.34e+24, "parameters": 3400000000},
      {"model": "Claude 2", "organization": "Anthropic", "date": "2023-07-11",
"compute": 3.87e+24, "parameters": null},
      {"model": "Qwen-Max", "organization": "Alibaba", "date": "2023-09-12",
"compute": 6e+24, "parameters": null},
      {"model": "Gemini 1.0 Ultra", "organization": "Google", "date": "2023-12-06",
"compute": 5e+25, "parameters": null},
      {"model": "GLM-4", "organization": "Zhipu AI", "date": "2024-01-16",
"compute": 1.2e+25, "parameters": null},
      {"model": "DeepSeek-V2", "organization": "DeepSeek", "date": "2024-05-06",
"compute": 5e+24, "parameters": 2360000000},
      {"model": "Nemotron-4 340B", "organization": "NVIDIA", "date": "2024-06-14",
"compute": 1.8e+25, "parameters": 3400000000},
      {"model": "Claude 3.5 Sonnet", "organization": "Anthropic", "date": "2024-06-20",
"compute": 2.7e+25, "parameters": null},
      {"model": "Llama 3.1 405B", "organization": "Meta", "date": "2024-07-23",
"compute": 3.8e+25, "parameters": 4050000000},
      {"model": "Grok-3", "organization": "xAI", "date": "2025-02-17", "compute": 3.5e+26, "parameters": 3e12},
      {"model": "Grok-4", "organization": "xAI", "date": "2025-07-09", "compute": 5.00e+26, "parameters": 3e12},
    ]
  }
}
```

```

    {"model": "Qwen3-Max", "organization": "Alibaba", "date": "2025-09-05",
"compute": 1.5e+25, "parameters": 1.0e12},
    {"model": "GPT-4.5", "organization": "OpenAI", "date": "2025-02-27",
"compute": 3.8e+26, "parameters": 1.0e10}
  ]
},
"layer": [
  {
    "mark": {"type": "circle", "opacity": 0.7},
    "encoding": {
      "x": {
        "field": "date",
        "type": "temporal",
        "title": "发布时间",
        "axis": {"format": "%Y", "grid": true}
      },
      "y": {
        "field": "compute",
        "type": "quantitative",
        "scale": {"type": "log"},
        "title": "训练计算量 (FLOPs, Log Scale)",
        "axis": {"format": ".0e", "grid": true}
      },
      "color": {
        "field": "organization",
        "type": "nominal",
        "title": "机构"
      },
      "size": {
        "field": "parameters",
        "type": "quantitative",
        "title": "参数量",
        "scale": {"range": [50, 500], "domain": [1e9, 3e12]},
        "legend": {"format": ".1s"}
      },
      "tooltip": [
        {"field": "model", "title": "模型"},
        {"field": "organization", "title": "机构"},
        {"field": "date", "type": "temporal", "title": "发布时间", "format": "%Y-%m-%d"},
        {"field": "compute", "title": "计算量", "format": ".2e"},
        {"field": "parameters", "title": "参数量", "format": ",.0f"}
      ]
    }
  },
  {
    "mark": {"type": "text", "align": "left", "dx": 8, "dy": 2, "fontSize":
10},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "compute", "type": "quantitative"},
      "text": {"field": "model"},
      "color": {"value": "#444"}
    }
  }
]

```



/// caption 图 1.1：顶级生成式 AI 模型训练算力需求演进趋势（2020-2025） ///

与此同时，模型架构本身也在向更「通信密集」的方向演进。特别是 2024–2025 年间，以 DeepSeek-V2/V3 为代表的 MoE（Mixture of Experts）架构在前沿模型中全面铺开，从根本上改变了 Scale-Up 域的通信需求画像。

**从 TP 主导到 EP 主导：**Dense 模型时代的核心通信模式是张量并行（TP）驱动的 AllReduce/ReduceScatter，其流量模式相对规则（集合操作、同步屏障）。MoE 架构引入的专家并行（EP）则要求高频 All-to-All 通信——每个 token 需要被路由到少量专家，且路由结果高度动态，流量模式呈现**不可预测的稀疏多播**特征。

| 维度       | Dense/TP 时代          | MoE/EP 时代             |
|----------|----------------------|-----------------------|
| 主要通信模式   | AllReduce, AllGather | All-to-All（动态路由）      |
| 流量可预测性   | 高（同步、对称）             | 低（稀疏、动态、非对称）          |
| 带宽敏感性    | 高（梯度同步）              | 极高（每层每 token 均需路由）    |
| 延迟敏感性    | 中（可隐藏）               | 极高（EP 路由在关键路径上）       |
| 尾延迟容忍度   | 可接受                  | 极低（一个慢专家拖慢全局）         |
| 对交换芯片的要求 | 聚合带宽                 | 聚合带宽 + 动态负载均衡 + 低排队时延 |

这不仅推高了总算力需求，还对互联带宽、延迟和动态调度能力提出了远超传统架构的要求。这一范式跃迁直接解释了为什么近期出现了如此密集的 Scale-Up 协议竞赛——EP 通信对带宽、延迟和动态调度能力的三重极端要求，使得传统网络的性能天花板成为模型训练效率的核心瓶颈，倒逼产业界进行全栈重构。

## AI for Science

除大模型训练外，AI for Science（AI4S）构成了第二条独立的算力指数需求曲线，且其驱动逻辑更为深层【归纳】。



科学发现正面临系统性的边际收益递减。Park et al. (2023) 对过去六十年间 4,500 万篇论文和 390 万项专利的分析表明，科研成果的“颠覆性”——即打破既有知识框架、开辟新方向的可能性——在各主要学科中持续下降[<sup>^</sup>disruption]。Bloom et al. (2020) 的经济学研究更直接指出，维持过去的创新速度所需的研发投入在急剧增加：例如，实现摩尔定律中芯片密度定期翻倍，所需的研究人员数量已是 1970 年代的 18 倍[<sup>^</sup>ideas]。制药行业的“Eroom 定律”则表明，每十亿美元研发经费所能诞生的新药数量约每 9 年减半[<sup>^</sup>eroom]。

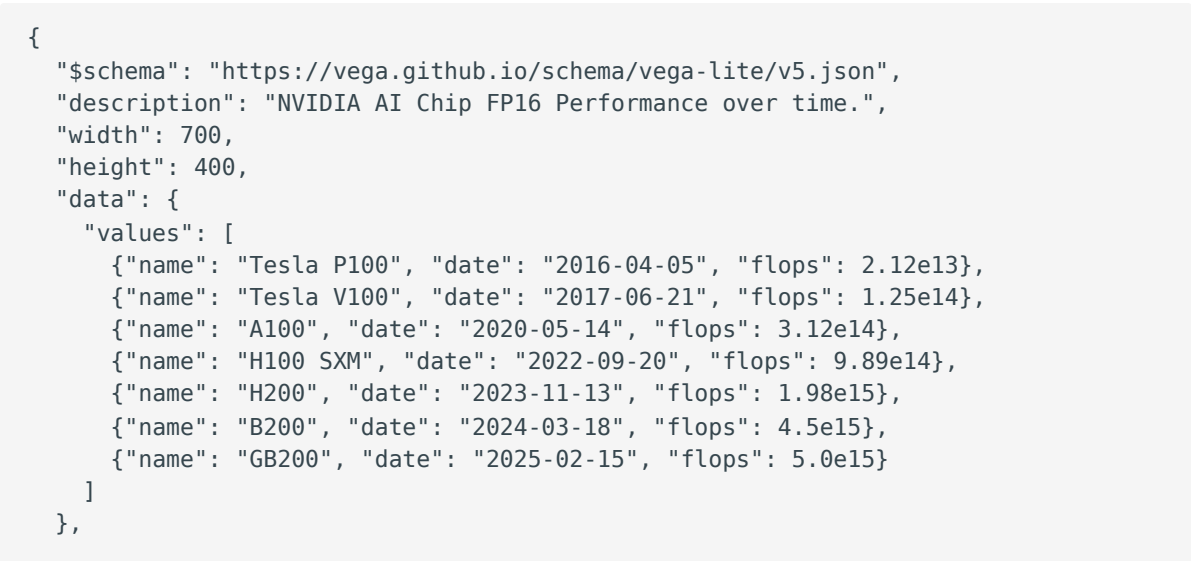
这些证据指向同一个结论：**科学探索的难度在上升，传统投入的边际收益在递减。仅依靠线性增加的人力、资金和时间来换取科研产出，正变得事倍功半【归纳】**。近期的标志性案例（如 AlphaFold 系列以计算预测替代传统实验方法）表明，打破这一困境的路径是将科研过程本身进行指数级 Scaling——通过 LLM 智能体批量执行实验以指数提升假设验证速度，通过自动化实验平台和高通量筛选以指数降低单次实验成本，通过科研 Copilot 系统性扩展研究人员的认知边界。这三个方向无一例外都转化为对算力基础设施的指数级需求【归纳】。

大模型的 Scaling Law 根植于参数-性能的幂律关系，AI4S 的 Scaling Law 根植于科学发现的认识论困境【归纳】。两者从完全不同的根源出发，却汇聚到同一个供给侧压力上：**算力基础设施必须以指数速度增长【归纳】**。即使其中一条曲线趋于饱和，另一条仍会持续施压。这使得对系统级算力指数增长的需求具有高度鲁棒性——它不依赖于某一个模型架构或某一个应用领域的繁荣【研判】。

## 系统算力的增长之谜

面对需求侧的双重指数压力，供给侧实际表现如何？

以 NVIDIA 数据中心训练 GPU 为标尺，从 V100（2017）到 GB200（2025），单芯片 FP16 张量峰值算力从 125 TFLOPS 攀升至 5,000 TFLOPS，8 年间增长约 40 倍【事实】。折算为两年一个代际周期，单芯片算力约每两年提升 2.5 倍【验证】。然而，若以系统级指标（如 DGX/NVL 超节点平台的整体训练吞吐）计算，每一代的系统级性能增幅则达到了惊人的 5-6 倍甚至更高【验证】。



```

"layer": [
  {
    "mark": {"type": "circle", "size": 60, "tooltip": true},
    "encoding": {
      "x": {"field": "date", "type": "temporal", "title": "发布时间"},
      "y": {
        "field": "flops",
        "type": "quantitative",
        "scale": {"type": "log"},
        "title": "FP16 算力 (FLOPs, Log Scale)",
        "axis": {"format": ".0e"}
      },
      "tooltip": [
        {"field": "name"},
        {"field": "flops", "format": ".2e"},
        {"field": "date", "type": "temporal", "format": "%Y-%m-%d"}
      ]
    }
  },
  {
    "mark": {"type": "text", "align": "left", "dx": 5, "dy": -5},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "flops", "type": "quantitative"},
      "text": {"field": "name"}
    }
  },
  {
    "transform": [
      {
        "regression": "flops",
        "on": "date",
        "method": "exp"
      }
    ],
    "mark": {"type": "line", "color": "firebrick", "strokeDash": [4, 4]},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "flops", "type": "quantitative"}
    }
  }
]
}

```

/// caption 图 1.2: NVIDIA 数据中心 GPU FP16 峰值算力演进趋势 ///

这个速度远超半导体制程本身能解释的范围【归纳】。同一时期，TSMC 从 12nm 演进至 4nm，晶体管密度约提升 4 倍【事实】。对应到同面积下的算力增益，即使叠加微架构改进，制程单独能贡献的部分不超过每两年约 2 倍【验证】。

需求侧要求指数增长，单芯片层面的制程红利与微架构演进每代只能提供约 2.5 倍。但超节点系统实际做到了 5–6 倍。**这额外的系统级红利从何而来？**

要回答这一问题，需先审视约束。限制 AI 芯片与系统性能的物理边界，可以归结为三面墙：

1. **物理空间边界（面积墙）**。受限于光刻机的掩膜版极限（约 858 mm<sup>2</sup>），单颗芯片上的晶体管数量是有绝对上限的。要获取更大的算力总量，必须走向多 Die 封装或多芯片互联——但这会引入片间通信开销。
2. **热力学边界（能耗墙）**。晶体管翻转就会发热。在给定的面积内塞入更多晶体管或提高频率，会导致功耗密度呈显著的非线性上升。如果热量散不出去，芯片就会面临热节流甚至烧毁（暗硅效应）。更严苛的是，互联通道本身也消耗功率：每条 112G SerDes 通道约消耗 5–8 pJ/bit，在 NVL72 规模的 130 TB/s 系统中，仅互联 I/O 功耗就可达千瓦量级。
3. **信息传输边界（带宽墙）**。芯片内部的算力密度增长始终快于片外数据供给能力。HBM 堆叠层数与通道数有封装极限，PCIe/SerDes 引脚数受制于基板面积与信号完整性。系统互联带宽更是受限于物理距离、线缆密度与光电转换代价。算力再高，如果单卡数据“喂不饱”，或者多卡之间“协同太慢”，晶体管就会空转。

这三面墙彼此耦合、此消彼长：要提升算力密度，就需要更多晶体管（撞面积墙）和更多数据供给（撞带宽墙）；要提升互联带宽，就需要更多 SerDes 通道和更高速率（撞功耗墙）；要控制功耗，就得限制频率或利用率，算力密度随之下降【归纳】。系统设计本质上是一个**多目标优化问题**——算力密度、互联带宽、内存容量、能效比、可扩展规模之间存在由物理定律施加的结构性冲突【归纳】。

在 AI 大规模训练场景中，三面墙的交织直接转化为三个不可避免的系统瓶颈：

- **互联带宽的“剪刀差”**：NVLink 等加速器间互联带宽与 PCIe/外设网络带宽存在数量级差异；当关键通信路径被“挤回”传统网络时，系统可扩展性会快速恶化。
- **“显存墙”与模型并行**：规模化训练不仅需要算力，更需要**有效的显存容量**和极低的并行通信代价。单卡显存上限更频繁地成为瓶颈，高带宽互联是支撑张量并行（TP）与专家并行（EP）、实现显存池化的前提。
- **通信时延与尾延迟敏感**：大规模集群中集合通信高频发生。跨机网络处于微秒级时延且极易受拥塞影响；而机柜级互联可将关键通信压缩到**亚微秒甚至百纳秒量级**，提供更可控的拥塞行为。

这三个瓶颈共同指向同一个系统事实：今天真正昂贵的，越来越不是再多做一次矩阵乘法，而是把高频、高价值、强耦合的数据交换安全地推出局部域外。也正因为如此，超节点的使命不是简单把更多芯片拼在一起，而是在“比特搬运比 FLOPs 更昂贵”的时代，尽可能重构并扩大计算的局部性边界。换句话说，**Scale-Up** 的价值不在于链路指标本身，而在于把尽可能多的关键通信稳定留在一个低时延、低抖动、故障域可控的受控域内【归纳】。而理解如何突破这些约束实现代际跃迁，需要一个能够描述多目标权衡的分析工具。

# 帕累托前沿与代际跃迁

描述多目标约束下可行权衡集合的数学工具是帕累托前沿（Pareto Frontier）【事实】。在多目标优化理论中，帕累托前沿是所有“非支配解”的集合——在这些解上，不可能在任何一个目标维度上取得改善，而不在至少一个其他维度上付出代价【事实】。

将这个框架映射到超节点设计，关键在于区分两种本质不同的操作：

- **沿前沿移动**：在同一技术世代内，设计者可以在前沿上选择不同的权衡点——比如牺牲互联带宽来换取更大的内存容量，或者牺牲能效来追求更高的峰值算力。这是设计哲学的选择，不改变可达的极限。
- **推移前沿**：引入一种此前不存在于设计空间中的新技术或新维度，使得原本不可能同时达到的性能组合变为可能。这才是代际跃迁的本质。

制程微缩带来的约 2 倍增益，本质上是在既有设计维度内把帕累托前沿向外推了一小步。但 NVIDIA 每一代做到的远不止于此。回顾从 Pascal/Volta 到 Blackwell 的四次代际跃迁，可以看到一个相对稳定的模式：每一代的核心创新都在向设计空间中引入一个**此前不存在或此前不占主导地位的变量**，从而在特定约束方向上大幅外推帕累托前沿：

| 代际跃迁                      | 引入的破局变量                               | 突破的约束墙       | 前沿外推方向                                                                                  |
|---------------------------|---------------------------------------|--------------|-----------------------------------------------------------------------------------------|
| Pascal → Volta (2017)     | Tensor Core<br>（专用矩阵乘法单元）             | 面积效率墙        | TDP 仅增 20%（250W→300W），AI 峰值算力跃升 6×                                                      |
| Volta → Ampere (2020)     | TF32 格式 + 2:4 结构化稀疏                   | 精度-算力权衡墙     | 无需改变硬件面积，等效算力翻倍；NVSwitch 2.0 将 8 卡互联带宽推至 600 GB/s/GPU                                   |
| Ampere → Hopper (2022)    | NVLink 4.0 + NVSwitch 3.0 + FP8 + TMA | 跨节点通信墙       | Scale-Up 域从单机 8 卡扩展至 256 卡 NVLink 域；FP8 与 TMA 解耦访存与计算                                   |
| Hopper → Blackwell (2024) | NV-HBI 双 Die 封装 + NVL72 铜缆背板 + FP4    | 面积墙 + 机柜级互联墙 | 10 TB/s D2D 实现逻辑单芯片化；72 卡 130 TB/s 近全互联；FP4 微张量缩放再压单比特算力（推理 30× / 训练 4×，TCO 与能耗均降低 25×） |

| 代际跃迁          | 引入的破局变量                  | 突破的约束墙      | 前沿外推方向           |
|---------------|--------------------------|-------------|------------------|
| 未来（Rubin 及以后） | HBM4 混合键合 + 光互联（CPO/LPO） | 内存能效墙 + 传输墙 | 继续推移互联距离与功耗的双重极限 |

Blackwell 的 NV-HBI（NVLink High Bandwidth Interface）值得特别关注。B200 GPU 由两颗 Die 通过 NV-HBI 以 10 TB/s 带宽互联，对软件呈现为一颗逻辑统一的 GPU——这本质上是对光罩极限（858 mm<sup>2</sup>）的工程突破：当单 Die 面积无法容纳所需的晶体管总量时，通过封装级 D2D 互联将“物理双芯”包装为“逻辑单芯”。NV-HBI 的成功工程落地，为整个行业的 Chiplet 路线提供了关键实证：只要 D2D 互联带宽足够高（量级达到 TB/s）、延迟足够低（< 5 ns），多 Die 封装就不仅是成本优化手段，更是突破面积墙后获取新一级算力密度的必要路径（参见第五章·多 Die 堆叠）。此外，Blackwell 还引入了 AI 驱动的 RAS Engine，可在运行时对 SRAM 和寄存器文件进行预测性诊断与预防性维护，将硬件 RAS 从“被动纠错”推向“主动预防”（参见第二章·RAS）。

每一代遵循相同的模式：**识别当代最紧的约束维度，引入一个新的设计变量来突破它【归纳】**。这恰好是帕累托前沿外推的操作定义。单芯片层面的制程微缩与架构演进是既有维度上的渐进推移，每代贡献了约 2.5 倍的算力增长；而系统级“破局变量”带来的超节点范围扩张则是新维度的跃迁式推移【归纳】。两者叠加，构成了系统级每代 5–6 倍甚至更高的综合吞吐增长【归纳】。

下图将这一“剪刀差”可视化。蓝色实线跟踪系统级训练吞吐的代际增长，灰色虚线跟踪单芯片 FP16 峰值算力的增长，红色标注的倍数即两者在每一代的差距。注意纵轴为对数刻度——在线性刻度下，这两条曲线的分叉会远比图中看起来更加剧烈。2025 年之后的虚线区域为趋势外推。



```

    "encoding": {
      "x": {"field": "x", "type": "temporal"},
      "y": {"field": "y", "type": "quantitative", "scale": {"type": "log"}},
      "text": {"value": "趋势外推 →"},
      "color": {"value": "#b45309"}
    }
  },
  {
    "data": {
      "values": [
        {"date": "2017-06-01", "chip": 1, "system": 1},
        {"date": "2020-05-01", "chip": 2.5, "system": 5},
        {"date": "2022-09-01", "chip": 7.9, "system": 30},
        {"date": "2024-03-01", "chip": 36, "system": 180},
        {"date": "2026-06-01", "chip": 90, "system": 1080}
      ]
    },
    "mark": {"type": "area", "opacity": 0.07, "interpolate": "monotone"},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "chip", "type": "quantitative", "scale": {"type": "log"},
      "domain": [0.6, 2500]}},
      "y2": {"field": "system"},
      "color": {"value": "#2563eb"}
    }
  },
  {
    "data": {
      "values": [
        {"date": "2017-06-01", "v": 1},
        {"date": "2020-05-01", "v": 5},
        {"date": "2022-09-01", "v": 30},
        {"date": "2024-03-01", "v": 180}
      ]
    },
    "mark": {"type": "line", "strokeWidth": 2.5, "interpolate": "monotone"},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "v", "type": "quantitative"},
      "color": {"value": "#2563eb"}
    }
  },
  {
    "data": {
      "values": [
        {"date": "2024-03-01", "v": 180},
        {"date": "2026-06-01", "v": 1080}
      ]
    },
    "mark": {"type": "line", "strokeWidth": 2.5, "interpolate": "monotone",
    "strokeDash": [8, 6]},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},

```

```

        "y": {"field": "v", "type": "quantitative"},
        "color": {"value": "#2563eb"}
    },
    {
        "data": {
            "values": [
                {"date": "2017-06-01", "v": 1},
                {"date": "2020-05-01", "v": 2.5},
                {"date": "2022-09-01", "v": 7.9},
                {"date": "2024-03-01", "v": 36}
            ]
        },
        "mark": {"type": "line", "strokeWidth": 2, "interpolate": "monotone",
"strokeDash": [6, 4]},
        "encoding": {
            "x": {"field": "date", "type": "temporal"},
            "y": {"field": "v", "type": "quantitative"},
            "color": {"value": "#94a3b8"}
        }
    },
    {
        "data": {
            "values": [
                {"date": "2024-03-01", "v": 36},
                {"date": "2026-06-01", "v": 90}
            ]
        },
        "mark": {"type": "line", "strokeWidth": 2, "interpolate": "monotone",
"strokeDash": [3, 5]},
        "encoding": {
            "x": {"field": "date", "type": "temporal"},
            "y": {"field": "v", "type": "quantitative"},
            "color": {"value": "#c0c8d4"}
        }
    },
    {
        "data": {
            "values": [
                {"date": "2020-05-01", "chip": 2.5, "system": 5},
                {"date": "2022-09-01", "chip": 7.9, "system": 30},
                {"date": "2024-03-01", "chip": 36, "system": 180},
                {"date": "2026-06-01", "chip": 90, "system": 1080}
            ]
        },
        "mark": {"type": "rule", "strokeDash": [2, 2]},
        "encoding": {
            "x": {"field": "date", "type": "temporal"},
            "y": {"field": "chip", "type": "quantitative"},
            "y2": {"field": "system"},
            "color": {"value": "#dc2626"},
            "strokeWidth": {"value": 1.2},
            "opacity": {"value": 0.5}
        }
    }

```

```

    }
  },
  {
    "data": {
      "values": [
        {"date": "2020-05-01", "midY": 3.54, "ratio": "2×"},
        {"date": "2022-09-01", "midY": 15.4, "ratio": "3.8×"},
        {"date": "2024-03-01", "midY": 80.5, "ratio": "5×"},
        {"date": "2026-06-01", "midY": 312, "ratio": "12×"}
      ]
    },
    "mark": {"type": "text", "fontSize": 15, "fontWeight": "bold", "align": "center"},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "midY", "type": "quantitative"},
      "text": {"field": "ratio"},
      "color": {"value": "#dc2626"}
    }
  },
  {
    "data": {
      "values": [
        {"date": "2017-06-01", "v": 1, "label": "DGX-1"},
        {"date": "2020-05-01", "v": 5, "label": "DGX A100"},
        {"date": "2022-09-01", "v": 30, "label": "DGX H100"},
        {"date": "2024-03-01", "v": 180, "label": "NVL72"},
        {"date": "2026-06-01", "v": 1080, "label": "Rubin NVL"}
      ]
    },
    "mark": {"type": "circle", "size": 60},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "v", "type": "quantitative"},
      "color": {"value": "#2563eb"},
      "tooltip": [{"field": "label", "title": "系统"}, {"field": "v", "title": "相对性能", "format": ".0f"}]
    }
  },
  {
    "data": {
      "values": [
        {"date": "2017-06-01", "v": 1, "label": "V100"},
        {"date": "2020-05-01", "v": 2.5, "label": "A100"},
        {"date": "2022-09-01", "v": 7.9, "label": "H100"},
        {"date": "2024-03-01", "v": 36, "label": "B200"},
        {"date": "2026-06-01", "v": 90, "label": "Rubin"}
      ]
    },
    "mark": {"type": "circle", "size": 40},
    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "v", "type": "quantitative"},

```



```

        "color": {"value": "#94a3b8"}
    }
},
{
    "data": {
        "values": [
            {"date": "2017-06-01", "v": 1, "label": "DGX-1", "note": "Tensor Core · NVLink 1.0"},
            {"date": "2020-05-01", "v": 5, "label": "DGX A100", "note": "TF32 · NVSwitch 全连接"},
            {"date": "2022-09-01", "v": 30, "label": "DGX H100", "note": "NVLink 4 · FP8 · 256 卡域"},
            {"date": "2024-03-01", "v": 180, "label": "NVL72", "note": "NV-HBI · 72-GPU · 液冷"},
            {"date": "2026-06-01", "v": 1080, "label": "Rubin NVL", "note": "NVLink 6 · HBM4 · 光互联"}
        ]
    },
    "layer": [
        {
            "mark": {"type": "text", "align": "left", "dx": 10, "dy": -12, "fontSize": 11, "fontWeight": "bold"},
            "encoding": {
                "x": {"field": "date", "type": "temporal"},
                "y": {"field": "v", "type": "quantitative"},
                "text": {"field": "label"},
                "color": {"value": "#1e40af"}
            }
        },
        {
            "mark": {"type": "text", "align": "left", "dx": 10, "dy": 4, "fontSize": 9},
            "encoding": {
                "x": {"field": "date", "type": "temporal"},
                "y": {"field": "v", "type": "quantitative"},
                "text": {"field": "note"},
                "color": {"value": "#6b7280"}
            }
        }
    ]
},
{
    "data": {
        "values": [
            {"date": "2017-06-01", "v": 1, "label": "V100"},
            {"date": "2020-05-01", "v": 2.5, "label": "A100"},
            {"date": "2022-09-01", "v": 7.9, "label": "H100"},
            {"date": "2024-03-01", "v": 36, "label": "B200"},
            {"date": "2026-06-01", "v": 90, "label": "Rubin"}
        ]
    },
    "mark": {"type": "text", "align": "right", "dx": -10, "dy": 4, "fontSize": 10},

```

```

    "encoding": {
      "x": {"field": "date", "type": "temporal"},
      "y": {"field": "v", "type": "quantitative"},
      "text": {"field": "label"},
      "color": {"value": "#64748b"}
    }
  },
  {
    "data": {"values": [{"x": "2017-02-01", "y": 1800}]},
    "mark": {"type": "text", "fontSize": 10, "align": "left", "fontStyle":
"italic"},
    "encoding": {
      "x": {"field": "x", "type": "temporal"},
      "y": {"field": "y", "type": "quantitative"},
      "text": {"value": "△ 纵轴为对数刻度，线性视角下剪刀差远比图中显示更大"},
      "color": {"value": "#9ca3af"}
    }
  }
],
"encoding": {
  "x": {"axis": {"title": "", "format": "%Y", "grid": false, "labelFontSize":
11}},
  "y": {"axis": {"title": "相对性能 (V100 / DGX-1 = 1×, 对数刻度)", "grid": true,
"gridOpacity": 0.12, "labelFontSize": 11, "titleFontSize": 12, "format": "~s"},
"scale": {"type": "log", "domain": [0.6, 2500]}}
},
"config": {
  "view": {"stroke": null},
  "axis": {"domainColor": "#e5e7eb", "tickColor": "#e5e7eb"}
}
}

```

/// caption 图 1.3：帕累托前沿外推的累积效应——系统级算力（蓝色实线）vs 单芯片算力（灰色虚线）增长对比。红色数字标注每一代的剪刀差倍数（系统增速 ÷ 芯片增速）。2025 年后为趋势外推。纵轴为对数刻度；在线性刻度下 2024 年的 5× 差距意味着系统级算力是芯片单独能解释部分的 5 倍。基线：V100 / DGX-1 (2017) = 1×。 ///

回顾这些“破局变量”，还可以发现一个趋势：它们的**物理作用尺度**在逐代扩大。Tensor Core 和 TF32 作用于**芯片内部**；NVLink Switch System 作用于**节点之间**；NV-HBI 作用于**封装级**；NVL72 铜缆背板作用于**整个机柜**。创新的物理尺度从芯片内部向系统外部持续扩展，帕累托前沿外推的“作用域”已经从单芯片扩大到了整个机柜乃至 Pod。

由此可得一个直接的工程推论：要继续维持高速的代际增长，仅在芯片层面做优化已远远不够。封装工艺、互联协议、网络拓扑、供电架构、液冷散热、系统软件——这些原本分属不同工程团队的技术，必须被放入同一个设计空间中进行联合优化。**超节点的本质不是“把更多的卡装进一个柜子”，而是“把足够多的设计维度纳入同一个可联合优化的工程边界内”。**

## 系统分层蓝图

从工程实现的角度，现代 AI 超算系统可以理解成一个从芯片内部一路往外放大的分层结构。下图用左右两列表示同一层内的两个对等实例，**横向箭头**表示同层之间的互联方式，**纵向箭头**表示从下一层组合到上一层所依赖的技术：**【归纳】**

```
block-beta
  columns 4

  PE0["处理单元 PE"] space:2 PE1["处理单元 PE"]
  space:4
  Die0["芯粒 Die"] space:2 Die1["芯粒 Die"]
  space:4
  Chip0["芯片 Chip"] space:2 Chip1["芯片 Chip"]
  space:4
  Node0["节点 Node"] space:2 Node1["节点 Node"]
  space:4
  Pod0["超节点 HBD"] space:2 Pod1["超节点 HBD"]
  space:4
  Cluster0["集群 Cluster"] space:2 Cluster1["集群 Cluster"]

  PE0<-- "NoC 片上网络" -->PE1
  Die0<-- "D2D 裸片互联" -->Die1
  Chip0<-- "NVLink 等 C2C" -->Chip1
  Node0<-- "NVLink/NVSwitch" -->Node1
  Pod0<-- "RDMA 网络" -->Pod1
  Cluster0<-- "数据中心网络" -->Cluster1

  PE0-- "NoC" -->Die0
  PE1-- "NoC" -->Die1
  Die0-- "Chiplet 封装" -->Chip0
  Die1-- "Chiplet 封装" -->Chip1
  Chip0-- "PCIe / C2C" -->Node0
  Chip1-- "PCIe / C2C" -->Node1
  Node0-- "Scale-Up 互联" -->Pod0
  Node1-- "Scale-Up 互联" -->Pod1
  Pod0-- "Scale-Out" -->Cluster0
  Pod1-- "Scale-Out" -->Cluster1
```

/// caption 图 1.4：AI 超算系统硬件分层蓝图——左右两列表示同层对等实例，横向为同层互联，纵向为层间组合 ///

- 1. **层级 1 - 芯粒内部 (Die)**：系统的最基本计算单元是处理单元 (PE)，例如 GPU 中的流式多处理器 (SM)。在单个硅片 (Die) 上，PE 通过片上网络 (NoC) 高效互联。
- 2. **层级 2 - 芯片 (Chip)**：借助先进封装技术，多个独立的芯粒被封装在一起。它们之间通过高速的 Die-to-Die (D2D) 接口通信，使其在逻辑上表现得像一个单片大芯片。

- 3. **层级 3 - 节点 (Node)**: 节点内的 GPU 之间通过芯片间互联 (C2C) 技术（如 NVLink）构建高速通信域。
- 4. **层级 4 - SuperPod/HBD**: 节点间以交换 Fabric（如 NVSwitch）组成机柜级 Scale-Up 域，这是超节点作用的核心边界。
- 5. **层级 5 - 集群 (Cluster)**: 多个 SuperPod 组合成集群。SuperPod 之间的通信依赖数据中心网络，使用基于 RDMA 的 Scale-Out 网络。

真正的竞争点往往集中在第 3–4 层：如何在机柜级保持低直径与高二分带宽，同时兼顾工程可运维性与软件可用性。

## 竞争格局

上述分析框架直接导出一个重要的竞争推论【研判】。

如果参与者 A 能够在每一代持续外推的是系统能力边界——不仅提高峰值指标，更能在制程、封装、互联、数值精度、软件栈、整机工程和恢复机制等约束下，把更多资源稳定维持在同一个受控协同域内——那么其系统算力可以维持约每两年 5–6 倍的增长【研判】。如果参与者 B 仅在芯片层面优化（制程微缩加架构演进，不具备超节点级系统集成能力），每两年约 2–2.5 倍【研判】。

\$N\$ 代之后，两者的系统性能差距以指数形式扩大。即使保守地取 A=5 倍、B=2.5 倍，差距比仍为  $2^N$ ：

| 代数 | 时间  | A 的累计增长 (5×/代) | B 的累计增长 (2.5×/代) | A/B 差距 |
|----|-----|----------------|------------------|--------|
| 1  | 2 年 | 5×             | 2.5×             | 2×     |
| 2  | 4 年 | 25×            | 6×               | 4×     |
| 3  | 6 年 | 125×           | 16×              | 8×     |
| 4  | 8 年 | 625×           | 39×              | 16×    |

四代之后差距是 16 倍。若取更激进的 A=6 倍、B=2 倍（即纯制程微缩，不含芯片级架构创新），差距在三代后即达 27 倍【验证】。无论取哪组参数，结论一致：一旦“**可持续交付的系统能力边界**”外推速度出现持续差异，这种差异就会以指数形式放大为系统性能差距【归纳】。这解释了为什么 AI 基础设施领域呈现出如此强烈的领先者优势——不是因为起跑线差距大，而是因为系统级创新速度的差异会在几代之内累积为数量级差距【归纳】。

然而，能够持续完成这种系统边界外推的参与者，在全球范围内屈指可数【归纳】。这不是因为缺乏某一项单点器件能力，而是因为边界外推要求多个设计维度同时成立——芯片与封装的迭代能力、Scale-Up 互联与交换能力、通信库与运行时、整机与液冷工程、开发者生态与客户牵引、大规模真实场景下的验证与恢复闭环——这些环节必须在同一个工程边界内联合推进【归纳】。当前能做到这种全栈垂直整合的，只有极少数体量足够大的平台型厂商【研判】。

对于多数参与者而言，约束往往不在某项单一技术缺失，而在于芯片、互联、封装、软件、整机、验证与生态尚未形成闭环【归纳】。单点能力有，系统闭环没有；器件能做，参考设计和整机交付弱；硬件能拼，软件兑现链路弱；有产品，没有方法论和跨产业协同接口。**差距的根源不是某个维度落后，而是缺乏把多个维度拉齐并联合优化的系统能力和协同机制【归纳】。**

这意味着对于不具备全栈垂直整合能力的参与者，真正的问题不在能否做出单点产品，而在于能否通过跨芯片、互联、封装、软件、整机与运维的产业协同，共同外推这条系统能力边界【研判】。这种协同的前提是：各方在共同的分析框架和评估语言下识别关键短板、对齐优先级、降低协调成本。也正因此，后续的帕累托分析框架、参考设计体系和 SPI 筹备构想才有意义：它们并不替代能力建设本身，而是为标准制定、验证平台搭建和联合项目提供共同起点【研判】。

指数级放大效应进一步说明，超节点能力不是"锦上添花"，而是关乎未来十年 AI 竞争力的基础设施能力【研判】。光互联、先进封装、Chiplet 和 Scale-Up 互联协议等环节，是需要优先建立自主能力的关键技术节点；帕累托前沿外推的分析框架和参考设计体系，可以为行业标准制定和跨组织协同提供方法论基础，降低各方"各说各话、各做各的"的协调成本；而没有系统级的前沿外推能力，算力基础设施建设将停留在"堆卡"阶段，无法实现算力的高效供给【研判】。

当然，硬件带宽和协议能力不会自动变成有效吞吐。下一章将转入软件系统，讨论统一寻址、通信运行时与 RAS 体系如何把这些硬件潜力稳定兑现为真实业务中的 Goodput。

[^scaling]: Kaplan, J. et al. "Scaling Laws for Neural Language Models." *arXiv:2001.08361*, 2020.  
[^disruption]: Park, M. et al. "Papers and patents are becoming less disruptive over time." *Nature*, 613, 138–144, 2023. [^ideas]: Bloom, N. et al. "Are Ideas Getting Harder to Find?" *American Economic Review*, 110(4), 1104–1144, 2020. [^eroom]: Scannell, J.W. et al. "Diagnosing the decline in pharmaceutical R&D efficiency." *Nature Reviews Drug Discovery*, 11, 191–200, 2012.

# 软件系统

上一章已经从系统架构层解释了，超节点为什么不能只停留在“更快的互联”上，而必须继续走向统一内存能力、Compute Fabric 以及计算平面与基础设施平面的分离。那一章回答的是**为什么需要这套东西**：为什么远端资源必须被重新组织成统一资源，为什么关键通信必须尽可能留在 HBD 内，为什么基础设施开销不能继续侵入主计算路径。但这些判断仍然停留在架构前提层面。真正决定超节点最终能否被交付、被使用、被持续扩展的，是下一步：**这些能力如何在真实系统中成立，又如何被稳定兑现为 Goodput**。

因此，本章不再重复上一章关于系统架构必要性的论证，而是沿着“统一访存”这条主线，转入**这些能力为什么会在软件层重新变成问题**。超节点软件不是驱动、库和框架的简单叠加，而是一套把远端资源重新组织成近似本地资源、再把这种“近似”控制在业务可接受范围内的软件体系。**硬件决定系统能力边界能推多远，软件决定这些被打开的能力能有多少被稳定兑现为 Goodput**。同样的互联带宽、同样的显存容量，在不同的软件栈与运行时策略下，最后可能呈现出完全不同的吞吐、尾延迟与可运维性。

开篇提出，超节点竞争的本质是系统能力边界外移速度之争，而这种外移本质上是一项系统工程：只有当封装、互联、内存、通信、调度与运维被纳入同一个可联合优化的工程边界时，新的设计变量才会真正转化为系统增益。软件系统在其中的作用，不只是“兑现硬件”，而是把远端资源带来的**位置差异、一致性代价、时序约束、隔离边界和恢复成本**压缩到业务仍愿意承受的范围内。也正是在这一层，许多原本看起来属于“硬件能力”的问题，开始转化为调度器、运行时、通信库和恢复机制必须共同承接的工程组织问题。

生态具有强惯性。平台能否被广泛采用，往往不取决于某个单点能力是否领先，而取决于它是否能够沿着既有编程模型、工具链和运行时资产继续演进。在这条路径中，内存语义不是局部实现细节，而是决定这条路径能否连续、生态能否延续、系统能力能否被持续组织起来的基础。统一内存也不是“能访问远端显存”这么简单，而是远端资源一旦真的进入统一调度视野，系统还能否把它的可见性、一致性、顺序成本、位置差异与恢复复杂度压缩到上层软件仍愿意把它当作稳定资源使用的程度。否则，能力虽然被打开，收益却会在软件层重新流失。

本章沿着这条线索展开。统一访存把这个问题讲得最具体：问题不在于能不能访问远端显存，而在于能否以可接受的一致性代价、地址复杂度和调度开销，把跨设备内存稳定兑现为 Goodput。更进一步看，统一内存语义的意义，不只是提升单次访问效率，更在于为软件生态提供一个可复用、可迁移、可持续演进的共同承载面。平台一旦不能被这套软件资产稳定承接，就必须反复支付适配与迁移成本；而一旦这种成本持续上升，原本被打开的系统能力就会重新退化为局部能力。

因此，前三节虽然仍沿着统一访存展开，但它们讨论的其实不是三个并列知识点，而是同一条兑现链上的三个层次：

- **语义层**：回答系统必须承诺什么。核心对象是 `load/store/atomic/fence`、可见性、顺序与一致性边界，讨论的是“远端资源在软件上究竟应被看成什么”。
- **机制层**：回答这些承诺靠什么成立。核心对象是 `GMMU`、`Fabric Address`、`IMEX`、事务代理、`SU Engine` 等，讨论的是“这些语义如何在一个真实系统中被组织起来”。
- **策略层**：回答能力如何变成收益。核心对象是冷热分层、放置、迁移、复用、隔离与回收，讨论的是“已经可访问的资源如何被持续调度成 `Goodput`，而不是被调度成本反过来吞掉”。

这三层之所以必须分开，是因为它们分别对应三种完全不同的失败方式：没有语义承诺，远端资源就不可能成为稳定资源；没有机制承接，抽象承诺就会停留在纸面；没有策略约束，已经成立的能力也会在迁移、碎片、抖动和恢复成本里重新失真。也只有把这三层切开，软件差距才不会被误读为“API 数量更多”或“机制名词更多”，而会重新回到系统能力究竟能否被持续兑现这个问题上。

在此基础上，后续三节把视野扩展到更广的软件栈。通信运行时与编程模型，讨论的是这些已经成立的资源如何被组织成可组合的协同行为；可观测性与 `RAS` 体系，讨论的是系统开始偏离边界时，软件如何重新把它拉回到可控范围；训练与推理系统工程，则是这一切的最终验收层，因为到了真实负载中，软件能力不再以模块形式出现，而会重新汇合为训练效率、推理尾延迟与长期可运维性这些系统结果。

从证据层面看，软件系统之所以必须被单独讨论，并不是因为“有了硬件之后总要再写一章软件”，而是因为超大规模集群的主要矛盾本来就已经转移到了协同与调度层。系统需要围绕“计算-存储-网络”做全链路资源联动与带宽适配；与此同时，异构算力统一调度、跨 `Pod` 分布式作业编排、训练与推理混部下的资源碎片控制，也已经成为决定资源利用率与服务稳定性的关键约束。这意味着，对超节点而言，软件系统不是锦上添花的运行时装，而是把硬件边界兑现为长期 `Goodput` 的主要控制层<sup>[^ultra-report-sw-cosched][^ultra-report-sw-scheduler]</sup>。也因此，网络在软件章节里不再适合作为一组脱离业务的局部指标来评价：对超节点真正有意义的，不是“单条链路有多快”，而是网络最终能为系统带来多少 `GPU` 利用率、多少长期 `Goodput`、多少训练效率提升，以及在推理场景下能否把尾延迟稳定压在可接受区间内。凡是无法回到这些系统级结果上的网络优化，最终都只能算局部改良，而不是超节点能力边界的真实外移。

但软件系统中的很多取舍并不能靠经验直接完成，需要通过建模、仿真和校准来量化“当前距离能力边界还有多远、哪个维度是最紧约束”。因此下一章将进入建模仿真，讨论如何把架构争论转化为可验证假设。

<sup>[^ultra-report-sw-cosched]</sup>: 云计算开源产业联盟，《超大规模智算集群关键技术及工程落地研究报告》，2025 年 12 月，第 18-19 页。报告将“算存网协同优化”定义为通过“计算-存储-网络”全链路资源联动与调度优化来提升集群整体效能。<sup>[^ultra-report-sw-scheduler]</sup>: 云计算开源产业联盟，《超大规模智算集群关键技术及工程落地研究报告》，2025 年 12 月，第 19-21 页、第 26-28 页。报告讨论了基于 `Kubernetes / Device Plugin` 的异构统一调度、跨 `Pod` 作业编排、资源碎片、长期运营稳定性与快速恢复等问题。

# 建模仿真

前两章分别回答了超节点的系统能力边界为何会被打开、以及这些被打开的边界如何通过软件栈被兑现。但对超节点产业而言，还剩下一个更基础的问题：**当不同路线都宣称自己更快、更省、更开放时，我们用什么尺度判断它们处在怎样的系统边界上？**

如果这个问题没有统一方法，后续关于互联、软件栈、参考设计和未来演进的判断，最终都容易退回到规格表比较或经验驱动的局部结论。建模仿真的意义，正是在于把这些离散、局部、易失真的观察组织成一张能够支撑方案判断的边界地图。本章围绕三个问题展开：

1. **为什么是边界比较**——单点 benchmark 和规格表对照为什么不够，多目标帕累托框架为什么更适合超节点决策？
2. **方法链条如何构建**——从负载建模到校准验证，六个环节各自做什么，行业工具基础覆盖到哪里？
3. **边界如何持续移动**——硬件代际、软件迭代和运维变化如何推动前沿外移，为什么建模仿真不能停在一次性结论上？

## 为什么是边界比较 {#pareto-definition}

在多目标优化中，如果方案 A 在所有目标上都不劣于方案 B，且至少在一个目标上严格优于 B，则称 A **支配** B。不被任何其他方案支配的方案即为**帕累托最优解**，所有帕累托最优解在目标空间中的投影构成**帕累托前沿**（Pareto Front）。前沿内侧是当前方案族的可达域，前沿外侧是尚不可达的理想域；边界本身，就是系统能力的上限地图。

对超节点级系统比较，目标向量通常包含但不限于以下维度：

| 目标维度 | 训练侧典型指标                      | 推理侧典型指标                      |
|------|------------------------------|------------------------------|
| 吞吐   | step time, samples/s         | tokens/s/GPU, tokens/s/MW    |
| 时延   | 迭代同步延迟, 气泡率                  | TTFT, TPOT, P95/P99          |
| 成本   | GPU-hours, \$/converged-loss | tokens/10k_rmb, \$/1M-tokens |
| 能效   | FLOPS/W, tokens/s/MW         | tokens/s/kW                  |



| 目标维度  | 训练侧典型指标              | 推理侧典型指标      |
|-------|----------------------|--------------|
| 可靠性   | Goodput / 理论峰值, MTTR | SLA 达标率, 可用性 |
| 工程复杂度 | 软件栈适配成本, 运维人效        | 部署周期, 混部管理开销 |

这些维度天然彼此拉扯。更高吞吐往往意味着更复杂的软件组织、更高功耗或更窄的部署条件；更低时延往往要为此付出成本和容量代价。因此，超节点选型的关键不再是“谁有一个最好的点”，而是“谁的边界更靠外、边界在什么条件下成立，以及应当在边界的什么位置取点”。

这一点在推理侧尤为明显。TP/PP、batch、量化、并行度、KV 策略和调度参数会迅速形成  $10^4$  量级的组合空间；延迟、吞吐、成本、显存、功耗和稳定性之间又天然冲突；突发流量、长尾分布和冷热模型混部则进一步让单点最优难以复用。帕累托分析真正提供的，不是一幅更复杂的图，而是一种“地图视角”：先剔除被支配区域，再分离可行域与不可行域，最后把不同团队的优化主张、线上观测和离线测试纳入同一坐标系比较。NVIDIA 在 Blackwell NVL72 白皮书中披露，仅 72 卡机柜上部署 1.8T MoE 模型，四个并行维度的组合就产生了超过 **2700 种候选配置**——不存在一种方案能在所有指标上同时取胜[<sup>icpe20-pareto-transfer</sup>]。

## 方法链条与工具基础 {#methodology}

一旦把问题改写为“判断边界”，方法论也会随之收束。超节点级比较不可能依赖逐点精算——真实候选空间往往在数千甚至上万量级，而高保真推演或端到端实测的单点成本足以让全空间穷举失去可行性。真正可行的路径只能是：**先用足够快的模型把边界大致描出来，再把有限的高成本验证资源压到最关键的区域。**

围绕这一目标，方法链条可以收束为六个连续环节：**负载模型、资源模型、系统模型、校准验证、输出接口、共演进闭环**。负载模型定义需求侧压力，资源模型定义供给侧边界，系统模型把两者压缩成多目标空间中的离散候选点，校准验证在关键区域收紧误差，输出接口把结果沉淀为可持续复用的判断对象，共演进闭环解释边界为何会随负载、软件、硬件和运维共同变化而持续移动。

公开研究和工程实践已经为前三个环节提供了重要基础。训练侧，ASTRA-sim 是目前最成体系的分布式训练仿真框架，其核心能力是将计算、通信和显存后端解耦，支持分层网络建模[<sup>astra-sim-1</sup>][<sup>astra-sim-2</sup>]；Chakra 定义了硬件无关、策略中立的负载表示格式[<sup>chakra</sup>]。自动并行方面，Alpa 等系统的成本模型能在数秒内评估候选配置的计算量、通信量和显存占用[<sup>alpa</sup>]。推理侧，Orca[<sup>orca</sup>]、Sarathi-Serve[<sup>sarathi</sup>]、Vidur[<sup>vidur</sup>]、Splitwise[<sup>splitwise</sup>]、DistServe[<sup>distserve</sup>] 已覆盖从到达建模、阶段拆分到调度优化的主要环节。

将这些工具映射到六个环节，可以看到哪些环节已有坚实基础、哪些仍是待填空白：

| 方法环节  | 已有工具覆盖                            | 覆盖程度 | 待补充方向                         |
|-------|-----------------------------------|------|-------------------------------|
| 负载模型  | Chakra 执行跟踪规约；<br>ASTRA-sim 计算图描述 | ★★★  | 超节点级多租户混合负载画像；推理侧到达模式与 SLA 分布 |
| 资源模型  | Alpa 成本模型；ASTRA-sim 分层网络后端        | ★★☆  | 国产互联拓扑参数库；能效与可靠性维度扩展          |
| 系统模型  | ASTRA-sim 端到端仿真；<br>Vidur 推理集群仿真  | ★★☆  | 训练/推理统一的多目标扫描引擎；帕累托前沿自动提取     |
| 校准验证  | 各工具分散的 validation 流程              | ★☆☆  | 统一校准基线与误差置信区间标准               |
| 输出接口  | —                                 | ☆☆☆  | 边界地图的标准化表示与版本管理               |
| 共演进闭环 | —                                 | ☆☆☆  | 负载-软件-硬件联动的边界漂移追踪机制           |

前三个环节已有可直接复用的工业级基础；后三个环节——校准验证、输出接口和共演进闭环——恰恰是把“仿真工具”升级为“持续服务超节点比较的分析能力”所必须补齐的系统性缺口。这里也需要特别强调：**评测不是独立主线，而是服务于校准验证**——真实测量的职责是在关键点上约束模型误差，而不是取代整个建模链条。

当六个环节成立后，建模仿真的结果就会沉淀为两类可持续复用的判断对象。第一类是**当前边界地图**：在统一负载集、统一指标口径和统一校准基线上，不同参考设计位于边界的什么位置、各自适用于怎样的负载画像。第二类是**边界变化假设集**：在已校准的当前边界之上，光互联、HBM4、Chiplet、MoE、长上下文等变量会把前沿向什么方向推、多大程度上缓解现有瓶颈、又会在何处引入新的约束。

## 前沿推移 {#pareto-frontier}

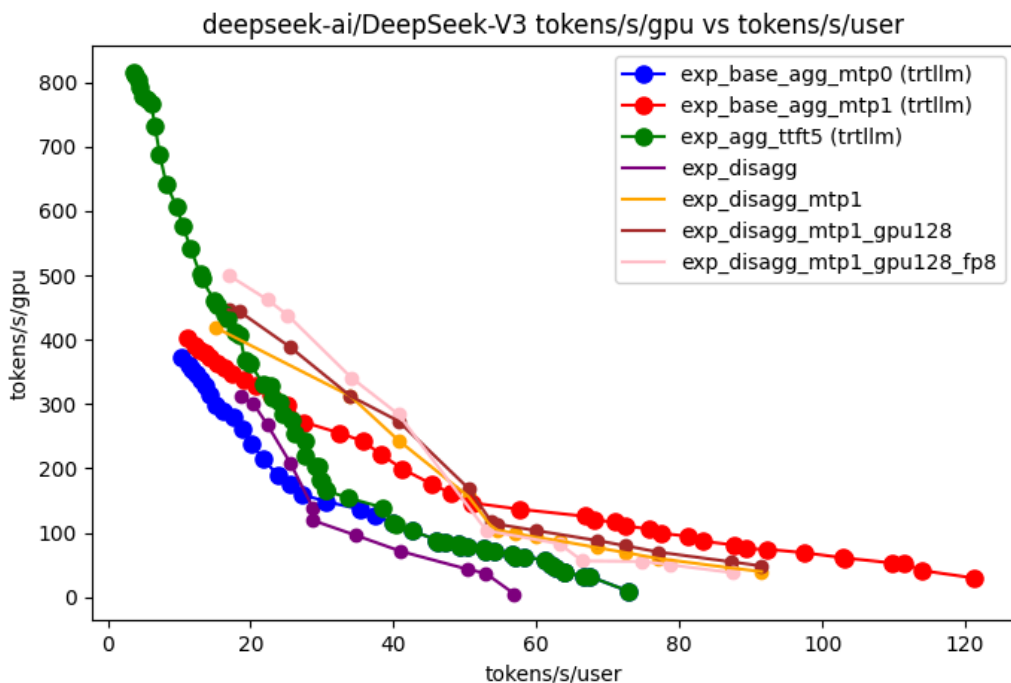
帕累托前沿不是静态资产，而是系统进步是否真实发生的直接度量。所谓“边界外推”，就是整条前沿在多个维度上向更优方向平移，它至少来自两个典型来源。

第一类来源是代际硬件跃迁。以推理场景的 `tokens/s/user` vs `tokens/s/MW` 坐标系为例，Blackwell NVL72 相比 Hopper NVL8 在甜点配置上实现了约 25–40× 的综合提升，而 Rubin 平台又进一步在吞吐密度和单位 token 成本上将前沿整体外推<sup>[^nvidia-rubin-sim]</sup>。InferenceX 开源基准在近千块 GPU 上的实测进一步拉大了这一数字：在 DeepSeek R1 大规模 MoE 推理场景下，GB300 NVL72 FP4 disagg+wideEP 相比 H100 FP8 disagg+wideEP 基线，perf/TCO 提升达 9.7×（40 tok/s/user）至 65×（116 tok/s/user）；若跨精度对比（FP4 vs FP8），实测差距甚至达到 100×<sup>[^inferenceX-v2]</sup>。训练侧同样如此：训练 10T MoE 模型达到相同目标，Rubin 仅需 Blackwell 约四分之一的 GPU 数量。代际跃迁改变的不是前沿上的取点，而是前沿本身的上界。

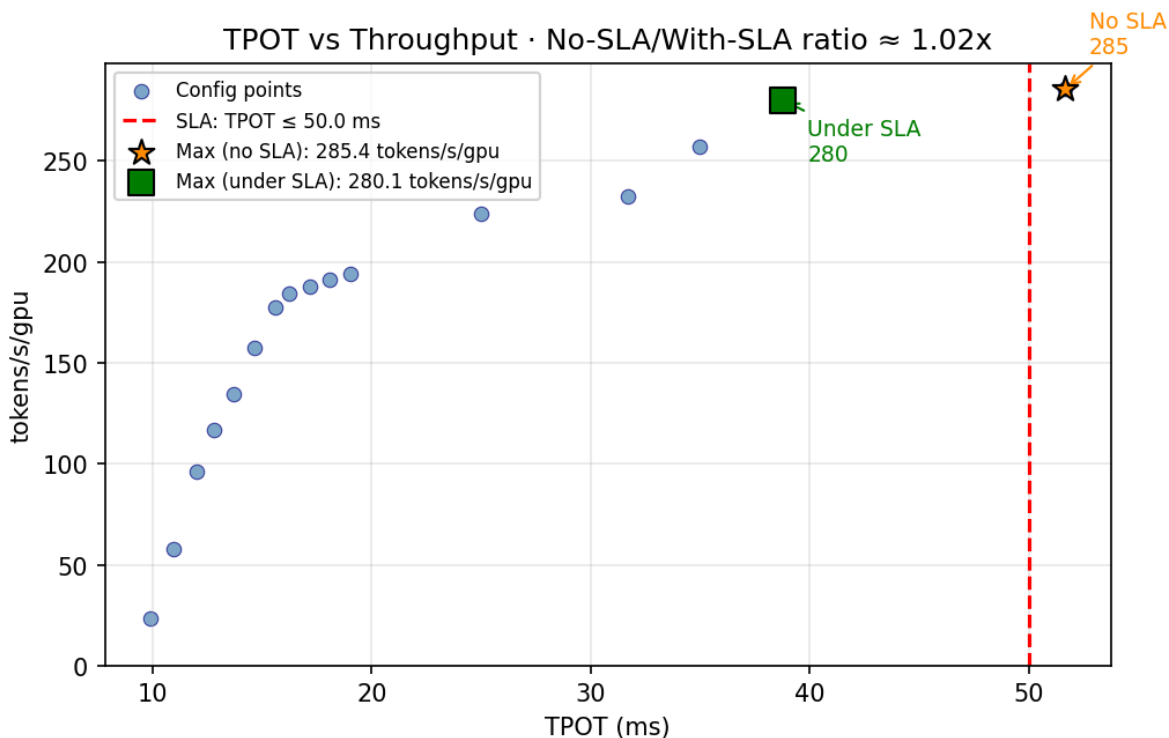
第二类来源是同代硬件上的软件迭代。NVIDIA 在 2025 年 8–10 月间的 InferenceMax 基准测试中展示了一个典型现象：在同一 GB200 NVL72 硬件上，仅通过数周的软件迭代，GPT-OSS 推理模型的帕累托前沿就被推出了近 5×<sup>[^nextplatform-pareto-sim]</sup>。这说明边界不能被视为一次测定后长期有效的稳态对象；如果软件栈、负载画像和运行条件持续变化，模型校准与边界更新就必须成为持续过程。

这种逻辑已经在[方法论验证案例](#)中得到具体展开。该案例在 L40S + A100 的异构 PD 分离架构上，通过 4 种候选架构、3 类请求场景以及多并发、多配置的系统级扫描，描出了帕累托前沿，再结合安全边际校准和成本敏感性分析，区分出哪些收益是稳健的、哪些只是局部改善。下表概括了几类典型优化路径对前沿的影响：

| 优化路径                                 | 前沿变化特征               | 代表性数据点                             |
|--------------------------------------|----------------------|------------------------------------|
| MTP（Multi-Token Prediction）          | 同硬件上沿吞吐轴外推，时延几乎不变    | tokens/s/GPU 提升 ~30–50%，TPOT 持平    |
| PD 分离（Prefill-Decode Disaggregation） | 打开新前沿分支：时延与吞吐可独立调优   | TTFT P99 降低 2–3×，吞吐密度提升 ~1.5×      |
| FP8 量化                               | 前沿整体平移：成本维度显著改善      | tokens/s/GPU 提升 ~1.8×，精度损失 <0.5%   |
| 规模扩展（Scale-out PD 节点）                | 沿前沿延伸但斜率递减：边际收益受网络约束 | 4 → 16 节点吞吐线性度 ~0.85，>32 节点降至 ~0.7 |



/// caption 不同技术路线对推理帕累托前沿的外推效应。MTP、PD 分离、FP8 量化和规模扩展不只是把某个点调得更好，而是在不同程度上把整条性能边界往外推。 ///



/// caption SLA 约束会直接改变可用前沿。红色虚线代表 TPOT  $\leq 50$  ms 的约束边界，它提醒我们：理论上更优的点，并不一定是工程上真正可交付的点。 ///

从更大的行业视角看，随着超节点从“把更多芯片连在一起”演进为“持续重写系统能力边界”的基础设施形态，建模仿真正在从辅助分析工具上升为一项基础能力：它不仅回答边界究竟在哪里、如何比较，也回答边界将如何继续移动。

[^nvidia-rubin-sim]: Kyle Aubrey. "Inside the NVIDIA Rubin Platform: Six New Chips, One AI Supercomputer." *NVIDIA Technical Blog*, Jan 2026.

[https://developer.nvidia.com/blog/inside-the-nvidia-rubin-platform-six-new-chips-one-](https://developer.nvidia.com/blog/inside-the-nvidia-rubin-platform-six-new-chips-one-ai-supercomputer/)

[ai-supercomputer/](https://developer.nvidia.com/blog/inside-the-nvidia-rubin-platform-six-new-chips-one-ai-supercomputer/) [^nextplatform-pareto-sim]: Timothy Prickett Morgan. "Software Pushes The AI Pareto Frontier More Than Hardware." *The Next Platform*, Oct 2025.

[https://www.nextplatform.com/ai/2025/10/21/software-pushes-the-ai-pareto-frontier-](https://www.nextplatform.com/ai/2025/10/21/software-pushes-the-ai-pareto-frontier-more-than-hardware/)

[more-than-hardware/](https://www.nextplatform.com/ai/2025/10/21/software-pushes-the-ai-pareto-frontier-more-than-hardware/) [^astra-sim-1]: Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. "ASTRA-SIM: Enabling SW/HW Co-Design Exploration for Distributed DL Training Platforms." ISPASS 2020. <https://ieeexplore.ieee.org/document/9238637/> [^astra-sim-2]: William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. "ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-model Training at Scale." ISPASS 2023. <https://arxiv.org/abs/2303.14006> [^chakra]: MLCommons Chakra Working Group. "Chakra Schema and Tools." 2023-.

<https://github.com/mlcommons/chakra> [^alpa]: Lianmin Zheng, Zhuohan Li, et al. "Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning." OSDI 2022.

<https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin> [^orca]: Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Sukjoon Lee, and Byung-Gon Chun. "Orca: A Distributed Serving System for Transformer-Based Generative Models." OSDI 2022.

<https://www.usenix.org/conference/osdi22/presentation/yu> [^sarathi]: Amey Agrawal, Nitin Kedia, Ashish Panwar, et al. "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve." OSDI 2024. <https://www.usenix.org/conference/osdi24/presentation/agrawal>

[^vidur]: Abhishek Vijaya Kumar, Giorgos Ananthanarayanan, et al. "Vidur: A Large-Scale Simulation Framework For LLM Inference." MLSys 2025. <https://arxiv.org/abs/2405.05465>

[^splitwise]: Pratyush Patel, Esha Choukse, et al. "Splitwise: Efficient Generative LLM Inference Using Phase Splitting." ISCA 2024. <https://arxiv.org/abs/2311.18677> [^distserve]: Yinmin Zhong, Shengyu Liu, et al. "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving." OSDI 2024.

<https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin> [^icpe20-pareto-transfer]: Pavel Valov, Jianmei Guo, and Krzysztof Czarnecki. "Transferring Pareto Frontiers across Heterogeneous Hardware Environments." ICPE 2020.

[https://research.spec.org/icpe\\_proceedings/2020/proceedings/p12.pdf](https://research.spec.org/icpe_proceedings/2020/proceedings/p12.pdf) [^inference-v2]: SemiAnalysis, "InferenceX v2: NVIDIA Blackwell Vs AMD vs Hopper", 2026-02.

<https://newsletter.semianalysis.com/p/inference-v2-nvidia-blackwell-vs>

## 参考设计

前文已经讨论了超节点最核心的技术问题：硬件系统能力边界如何打开，软件侧又如何将这些能力稳定兑现，也给出了分析和度量一套软硬件系统能力边界的方法。到了第四章，问题不再是“能力能否成立”，而是在什么约束下做什么取舍，**构建一套现实可交付的超节点系统。**

超节点系统的核心，不在于抽象地追求某个单项指标最优，而在于在供应链、产品稳定性、可运维性、软件复杂度与商业竞争压力的共同约束下，找到性能与成本的帕累托前沿，并在前沿上作出适合自己的取舍。对超节点来说，真正拉开构型差异的，始终是三件事：哪些高价值通信必须留在受控域内，哪些语义必须由硬件原生承担、哪些可以交给软件与控制面吸收，以及系统愿意为此承受多大的工程与产业代价。

## 五种构型

沿着这条判断线看，第四章讨论的五种构型，并不是五份并列铺开材料，而是五种不同的系统回答。它们共同支撑的，是同一个章节判断：**当前边界上不存在统一最优解，只存在在不同约束下各自不可支配的构型。**

- **标准总线方案（UALink/UB 为代表）**把强语义访问看作第一约束，核心目标是把更多远端资源继续组织成近似本地资源。它最接近前两章里“统一资源、强语义访问、Goodput 兑现”的主问题。
- **标准以太网方案（ESUN/OISA/ETH+ 等）**优先守住生态连续性、供应链连续性与运维连续性，在此基础上接受部分语义和时延让步，换取现实世界里更强的可交付性。
- **Dragonfly + OCS**把局部性边界从柜级继续外推到群级，同时尽量不重写全栈，因此更接近“规模继续外推，但软件与生态代价仍需可控”的构型。
- **3D Torus + OCS**把重点从统一交换语义转向切片可用性、故障绕行和拓扑弹性，更像是在重新组织受控域本身。
- **大环路 + dOCS**则把交换能力进一步下沉到端侧，甚至测试去交换化是否可能成为下一代高带宽域的默认组织方式；它对应的不是当前主流工程答案，而是对下一条前沿外推路径的激进押注。

## 构型比较

把这五种构型放到同一张表里，重要的不是谁的单项指标更突出，而是谁在多重约束下更接近当前可达边界。只有沿着同一套坐标比较，后面关于“标准”与“探索”的区分才不会沦为抽象分类，而会重新落回系统取舍本身。它们至少可以沿以下六个维度横向比较：

| 维度      | 标准总线           | 标准以太网   | Dragonfly+OCS | Torus+OCS | 大环路+dOCS |
|---------|----------------|---------|---------------|-----------|----------|
| 域内强语义保留 | 最强             | 中等      | 中等            | 中等偏强      | 潜力最高，未验证 |
| 规模外推    | Pod 内强，跨 Pod 弱 | 柜级到中等群级 | 千卡到万卡         | 千卡到万卡     | 长期潜力大    |
| 拓扑弹性    | 低              | 低       | 中             | 高         | 极高       |
| 生态与供应链  | 低              | 高       | 中             | 中低        | 低        |
| 软件门槛    | 高              | 低       | 中高            | 高         | 极高       |
| 长期能效    | 中              | 中       | 中高            | 高         | 极高       |

这张表最重要的结论，不是谁更优，而是**没有任何一列能够同时占优**。系统工程在这里承担的职责，不是消除这些冲突，而是把这些冲突压到当前可达边界上，再根据真实约束选择构型。所谓参考设计，本质上是在决定：哪些通信必须继续保留在受控域内，哪些能力可以交给软件和控制面吸收，哪些代价必须由供应链、运维体系和组织能力来承担。也正因此，所谓标准构型与探索构型，并不是名称上的划分，而是两种不同的边界推进方式。

标准构型更接近当前已经能够被工程体系稳定承接的一侧。它们并不意味着没有代价，而是这些代价更容易被现有供应链、软件栈和运维组织吸收，因此更适合作为当下产业条件下的主流落点。

- **标准总线方案**：优先把强语义访问留在受控域内，适合 TP、显存共享和细粒度同步场景。
- **标准以太网方案**：接受部分语义让步，换取供应链、运维和跨厂商协同的连续性。

探索构型则更接近对边界外推的主动尝试。它们不天然更先进，而是在某些维度上把系统推向更靠外的一侧，同时接受更高的控制面复杂度、更重的软件负担和更大的工程不确定性。它们的意义，不是立即替代标准答案，而是提前暴露下一阶段系统外推最可能遇到的真实约束。

- **Dragonfly + OCS**：把局部性边界从柜级推向群级，同时尽量保留既有以太生态与软件组织方式。
- **Torus + OCS**：把拓扑可重构性本身变成系统能力，用以提升切片可用性、任务贴合度与故障绕行能力。
- **大环路 + 分布式 dOCS**：进一步测试交换能力下沉与去交换化是否可能成为下一代高带宽默认组织方式。

## 协同工程

把这五种构型放在一起看，第四章最终落出的并不是一套“路线图”，而是一个更硬的结构事实：**超节点不是单产品竞争，而是跨芯片、互联、光学、封装、整机、软件与运维的协同工程**。芯片厂商决定算力与封装底座，交换与互联厂商决定局部性边界的硬件实现，光模块与整机厂商决定能效与交付形态，软件与框架团队则决定这些能力能否稳定兑现为 **Goodput**。

也正因如此，这些方案都不会静止不动。它们的成立条件，仍会被光互联、封装、**Chiplet**、**HBM/3D DRAM** 以及模型形态的演进持续改写。第四章只是把当前边界上的几种构型摆清；下一章讨论的，则是哪类变量最可能继续改变这条边界的形状。



# 未来演进

第四章给出了当前系统能力边界上的五个参考设计。但这些方案的成立条件并非静态——它们依赖的互联形态、封装路径、内存架构和负载画像都在变化。本章不再罗列未来技术，而是回答一个更直接的路线判断问题：**在未来 2–3 年内，哪些技术变量最可能以可工程化的方式改变超节点系统能力边界的主导约束，并因此重写当前参考设计的优先级？**

为什么是 2–3 年？因为 NVIDIA 的代际路线图提供了一条清晰的淘汰基准线：从 Hopper 到 Blackwell 到 Rubin，系统级推理性能每代提升约 10×（硬件 ~2× × 软件 ~5×），训练效率的 GPU 数量需求每代缩减至约 1/4。任何超节点架构选择，如果不能在这一窗口期内把系统能力边界向外推一个数量级，就会被下一代平台直接替代。因此，本章对每一项技术变量的讨论，都要落回两个判断：**它先改写什么约束？它会如何改变第四章的方案优先级？**

## 哪些变量会改写超节点能力边界

能力边界不只是“向外推”——它的坐标轴本身也在变化。当工作负载从稠密训练演进到推理-reasoning 混合、长上下文 MoE、多模态 Agent 时，“tokens/s/user vs tokens/s/GPU”不再是唯一的权衡面。新的关键维度正在浮现：能效（tokens/s/MW）、上下文容量（KV cache \$/token）、交互速率、以及可重构性（动态拓扑使能力边界从静态曲线变为动态曲面）。这并不意味着超节点设计可以被压缩成某一张二维图，而是意味着系统价值正在越来越多地由**单位功率下的有效产出、可承接的上下文规模、尾延迟与运维复杂度**共同定义。在这些新维度浮现的同时，三组技术变量正在改写既有约束：

**互联与拓扑：**当 SerDes 速率提升、链路预算趋紧时，铜互联在插损、功耗、可维护性上的压力会迅速放大。光互联与光交换的引入，本质上是在更大物理尺度上把“带宽与抖动”重新做可控化，同时为可重构拓扑提供物理抓手。LP0 / NP0 / CP0 也不是简单的“一代比一代好”，而是分别对应可插拔降功耗、近封装过渡、共封装极限带宽密度三种不同窗口；它们改变的也不只是链路器件，而是机架布线、供配电、冷却与维护半径的系统组织方式。它先改写的约束是**链路功耗、链路预算与拓扑灵活性**。

**封装与近存储：**算力密度的提升越来越依赖 2.5D/3D 集成与堆叠内存（HBM3e → HBM4 → 3D DRAM）。它们把互联距离压缩到封装内，却把供电、散热、良率与测试的复杂度推到台前。Chiplet + UCIe 的模块化路径在算力密度、良率、成本与互联带宽之间寻找工程可行解，也在重新划分“多少问题留在封装内解决、多少问题外推到板级和柜级”的系统边界。它先改写的约束是**算力密度与内存容量/成本**。

**模型与负载形态：**低精度（NVFP4）、长上下文、稀疏/MoE、多模态，会把系统压力从"峰值算力"转移到"内存带宽/容量、通信小包与尾延迟、以及调度与资源隔离"。模型技术的演进不是独立话题，而是用来定义互联、封装与系统软件的约束条件——它决定了当前能力边界上哪些区域是当下最需要优化的。它先改写的约束是**瓶颈位置本身**。

## 未来变量观察总览

未来变量不宜被写成一张带有强排序含义的“优先级表”。不同组织面对的约束并不相同，供应链位置、负载画像、交付周期和组织能力都会改变这些变量的实际先后顺序。更合适的呈现方式，是把**当前证据、产业动向、可能改写的约束以及**与第四章的关系并列放在一起。

下表中的“证据基础”分三档：**已验证趋势**（已有量产、部署或主流软件支持）、**工程验证中**（已有样机、原型或明确路线图）、**持续跟踪**（方向清楚，但产业化节奏仍不确定）。

| 技术变量      | 当前证据基础 | 产业动向与依据                                                                                  | 可能改写的约束            | 与第四章的关系                               |
|-----------|--------|------------------------------------------------------------------------------------------|--------------------|---------------------------------------|
| LPO 光模块   | 已验证趋势  | 公开资料和行业规范普遍把 LPO 视为短距降功耗、保留可插拔形态的现实部署路径，功耗和时延收益已具备较强工程证据[^future-lpo]                    | 链路功耗、插损、机柜级布线压力    | 对大多数构型都属于通用改善                         |
| NPO / CPO | 工程验证中  | NPO 作为 2024-2026 过渡窗口的定位更清晰；CPO 在交换芯片侧已有更明确的原型与产品化信号，但维护、热设计和测试体系仍是约束[^future-packaging] | 功耗密度、链路预算、集成度、可维护性 | 对探索构型和高密度以太构型影响更直接                    |
| OCS 光交换   | 工程验证中  | Google TPU 体系、MEMS/硅光 OCS 原型与分布式 dOCS 路线都在推进[^future-interconnect]                       | 拓扑灵活性、故障隔离、组间带宽配置  | 主要影响 Dragonfly + OCS、Torus + OCS、dOCS |

| 技术变量                  | 当前证据基础                  | 产业动向与依据                                                                   | 可能改写的约束               | 与第四章的关系              |
|-----------------------|-------------------------|---------------------------------------------------------------------------|-----------------------|----------------------|
| HBM4 / 3D DRAM        | HBM4 已验证趋势；3D DRAM 持续跟踪 | HBM4 已进入明确代际路线，3D DRAM 仍处于工艺与系统验证阶段[^future-hbm]                          | 节点内存容量、带宽/容量比、近存储成本   | 会重新分配节点内外的压力边界       |
| Chiplet + UCIe        | 工程验证中                   | UCIe 生态持续扩张，Chiplet 已从单厂商实践走向更广泛的接口协同[^future-chiplet]                    | 算力密度、良率、成本、D2D 带宽     | 对总线语义延伸和节点内高密度集成影响更大 |
| 低精度（FP8/FP6/FP4 等）    | 已验证趋势                   | Hopper/Blackwell 及主流框架持续推进低精度训练与推理[^future-low-precision]                 | 算力/带宽比、显存占用、数值稳定性管理   | 会重排所有构型下的瓶颈位置        |
| 长上下文 / MoE / 多模态      | 已验证趋势                   | 主流模型已普遍进入长上下文、MoE、多模态混合演进阶段[^future-model-tech]<br>[^future-long-context] | 内存层次、尾延迟、调度与隔离        | 会改变第四章各构型的适配负载分布     |
| 可重构拓扑控制面              | 持续跟踪                    | 产业界开始关注拓扑控制、任务切片与光路编排协同，但生产级框架仍有限[^future-interconnect]                   | 软件复杂度、调度弹性、恢复路径       | 决定探索构型能否从验证走向部署      |
| 软件栈演进（通信库 / RAS / 调度） | 已验证趋势                   | 同代硬件上，通信库、推理引擎、运行时与 RAS 的更新已持续改变 Goodput[^future-model-tech]              | Goodput 兑现度、部署效率、恢复成本 | 影响全部参考设计的真实交付结果      |

## 当前可以较确定看到的变化方向

与其直接判断“谁排第一”，更有价值的是先识别哪些变化已经发生、哪些进入工程验证、哪些仍应保持观察。

**已经在影响当前部署的变量**主要有三类。第一，低精度与模型形态变化已经在重排系统瓶颈，压力正从单纯追求峰值算力，转向更关注内存带宽、尾延迟和调度效率<sup>[^future-model-tech][^future-low-precision]</sup>。第二，LPO 已成为现实可部署的降功耗路径，至少在机柜级与短距场景下，它不再只是路线图概念<sup>[^future-lpo]</sup>。第三，软件栈的持续演进已经证明，同一代硬件上的 Goodput 会随通信库、运行时和推理引擎更新而显著变化，这类变量虽然不改变器件形态，却会改变今天方案比较的结果。

**已经进入工程验证窗口的变量**主要集中在 OCS、NPO/CPO、HBM4 与 Chiplet + UCle。产业路线已很清楚，但真正决定其影响强度的，仍然是器件成熟度、供应链就绪度、软件控制面和系统级良率，而不是实验室中的最佳数字。特别是在光互联方向上，NPO 更接近当前维护体系仍可承接的过渡窗口，CPO 则更接近在极致带宽密度目标下逐步逼近的下一层边界<sup>[^future-interconnect][^future-packaging][^future-chiplet][^future-hbm]</sup>。

**仍应保持观察的变量**主要是 3D DRAM、分布式 dOCS 与生产级可重构拓扑控制面。它们都很可能成为下一代系统边界的重要变量，但当前更适合被视为需要持续记录拐点条件的方向，而不是当前部署的前提假设。

第三章的方法论在这里的作用，不是替这些变量给出确定性结论，而是帮助建立三类量化对象：已经可校准的趋势、可以做情景推演的工程验证项、以及需要设定监测阈值的前沿方向。

## 对第四章参考设计的观察性影响

从当前可见的产业动向出发，单一路线结论仍然过早。更有意义的是并列展示“哪些构型更可能受益”“哪些前提仍需观察”。

| 技术变量            | 更可能受益的构型                          | 仍需观察的条件                   |
|-----------------|-----------------------------------|---------------------------|
| LPO / NPO / CPO | 标准以太网、Dragonfly + OCS、Torus + OCS | 功耗改善是否能覆盖维护复杂度与供应链改造成本    |
| OCS 光交换         | Dragonfly + OCS、Torus + OCS、dOCS  | 控制面延迟、故障恢复时间、运维可用性是否达到生产级 |
| HBM4 / 3D DRAM  | 标准总线、标准以太网、长上下文相关构型               | 容量/成本曲线、热边界、良率与分層管理开销     |

| 技术变量           | 更可能受益的构型      | 仍需观察的条件                       |
|----------------|---------------|-------------------------------|
| Chiplet + UCIe | 标准总线、节点内高密度构型 | UCIe 生态、D2D 透明性与协议兼容性是否成熟     |
| 低精度与模型形态变化     | 全部构型          | 不同负载画像下，瓶颈是否从算力进一步转向内存、尾延迟与调度 |
| 可重构拓扑控制面       | 探索构型          | 控制框架是否能稳定承接切片、路由和恢复逻辑         |
| 软件栈演进          | 全部构型          | Goodput 提升能否被持续复现，而不是一次性优化结果  |

从今天的产业状态看，第四章“成熟路径优先、探索路径并行验证”的总方向仍然成立，但其依据应当更多来自证据强弱和工程窗口，而不是单一路线判断。更自然的分层是：

- **当前可直接吸收的变量：**LPO、低精度、软件栈演进。
- **适合并行验证的变量：**OCS 控制面、HBM4/更高层次近存储、Chiplet + UCIe 兼容性。
- **更适合持续跟踪的变量：**3D DRAM、dOCS、生产级可重构拓扑框架。

## 建议持续跟踪的判断指标

真正能改变结论的，仍然是少数几个持续变化的指标：

- **OCS / dOCS 的控制面时延与恢复时间：**决定探索构型能否从验证走向生产。
- **CPO / NPO 的功耗密度与维护成本：**决定高密度构型是否真的具备系统级优势。
- **HBM4 / 3D DRAM 的容量/成本与热设计边界：**决定更多压力是否会重新回到节点内解决。
- **主流负载中长上下文、MoE、多模态的占比变化：**决定第四章  $w1/w2/w3$  的权重是否需要调整。
- **软件栈承接新硬件能力的速度：**决定“硬件打开上限、软件决定落地”这一判断在未来窗口中的兑现程度。

本章各子节将沿上述观察框架展开，分别讨论互联与光交换、封装与 Chiplet、HBM 与 3D DRAM、以及模型形态演进的工程细节与系统取舍。每一节都将回到同一个收束问题：**哪些约束已经在变化，哪些变化值得进入当前部署判断，哪些变化仍需作为未来窗口持续跟踪。**

[^future-ipo]: 参见[互联技术演进](#)与[先进封装技术演进](#)中关于 LP0 的量产形态、短距场景与产业部署讨论。[^future-packaging]: 参见[先进封装技术演进](#)中关于 NP0/CP0 的系统级权衡、产业路线与工程挑战。[^future-interconnect]: 参见[互联技术演进](#)中关于 LP0、0CS、d0CS 与不同规模互连路径的讨论。[^future-hbm]: 参见[堆叠内存技术（HBM/3D DRAM）](#)中关于 HBM4、3D DRAM、混合键合与容量/成本边界的讨论。[^future-chiplet]: 参见[多Die堆叠技术（Chiplet 与 UCle）](#)中关于 Chiplet + UCle 的工程定位、D2D 门槛与产业路径。[^future-model-tech]: 参见[先进模型技术演进](#)中关于 MoE、多模态、模型形态变化如何重排系统瓶颈的讨论。[^future-low-precision]: 参见[低精度数值格式](#)中关于 FP8/FP6/FP4、通信与内存配比变化的讨论。[^future-long-context]: 参见[超长序列技术](#)中关于 KV Cache、长上下文与内存层次压力的讨论。

## 结束语

本白皮书从一条因果链出发展开全部讨论：需求侧的双重指数增长不可逆转，供给侧必须维持远超制程红利的系统级增速，而这要求每一代持续推动系统能力边界外移；超节点正是使这种外移在机柜尺度上成为可能的工程形态。六章内容分别完成了这条因果链的定义、兑现、度量、选择、预判与回收。

## 技术结论

### 全书最核心的判断

回顾前文，可以收束为五个判断：

- **系统级"2 年 5–6 倍"的增速不是摩尔定律的延续，而是系统能力边界持续外移的累积结果。**每一代的核心创新都是向设计空间中引入此前不存在的变量（Tensor Core、NVSwitch、NV-HBI、NVL72 铜缆背板），从而使原本不可同时达到的一组指标组合变得可达。单芯片制程红利只贡献了约 2.5 倍，其余的系统级乘数来自超节点范围内的多维联合优化。
- **超节点的本质不是"把更多的卡装进柜子"，而是"把足够多的设计维度纳入同一个可联合优化的工程边界"。**带宽、时延、规模、功耗、成本、软件复杂度和可运维性共同决定系统能力边界。为了形式化描述这条边界，白皮书使用帕累托前沿作为分析语言。
- **硬件打开可能性，软件决定路径连续性与兑现度。**统一内存、通信运行时和 RAS 的意义，不是单独提供新能力，而是把这些被打开的系统能力组织成一条可持续演进的软件路径，使新变量能够沿着既有生态被持续吸收，并最终稳定兑现为 Goodput。
- **未来技术演进的意义，在于改写系统能力边界的维度与形状。**光互联、先进封装、Chiplet、HBM/3D DRAM 与模型形态变化，不是平行专题，而是在持续重新定义"今天该怎么设计"的约束条件。
- **开放标准与系统扩展边界同样需要被放在全栈层面理解。**若没有统一通信抽象、统一接口和统一运维能力，开放互联仍可能在软件栈和工具链层重新碎片化；而随着跨中心协同和算网一体化成为现实需求，Scale-Across 也会越来越多地决定哪些局部最优能够继续扩展为系统最优。

### 竞争路径：边界外移速度决定差距

第一章的分析框架导出一个重要推论：系统能力边界外移速度的差异，会以指数形式放大为系统性能差距。能在制程、封装、互联、精度和软件栈上同时引入新设计变量的参与者，系统算力每两年增长 5–6 倍；仅在芯片层面优化的参与者，每两年约 2–2.5 倍。四代之后差距即达 16 倍。

这意味着超节点竞争的本质不是比拼任何单一规格，而是比拼**系统能力边界外移的速度**。当前能够独立完成这种全栈联合优化的，只有极少数垂直整合型平台。对多数参与者而言，约束往往不在某项单一技术缺失，而在于芯片、互联、封装、软件、整机与验证尚未形成系统闭环。本白皮书提供的帕累托分析框架、参考设计体系和 SPI 筹备说明，并不替代这种能力建设本身，而是为产业协同提供统一的分析语言、比较坐标和问题清单，帮助各方识别关键短板、对齐优先级，并为后续标准制定、验证平台和联合项目降低协同成本。其中软件部分需要强调的，不是“补齐若干模块”，而是围绕内存语义、通信语义、运行时与运维体系形成一条能够持续吸收新变量的演进路径；这条路径能否连续，本身就是系统竞争力的一部分。

## 面向不同读者的启示

以上技术结论对不同读者有不同的实践含义。以下按受众分别展开。

### 对技术决策者：当前阶段的方案选择

[第五章](#)已经把未来 2–3 年的关键技术变量按证据等级、时间窗和对参考设计的影响做了优先级排序，并给出了“近期主力 / 并行验证 / 中期储备”的部署分层。这里只提炼对技术决策者最直接的三条建议：

- **标准构型** 仍应作为当前阶段的主力路径——LPO、HBM4 和低精度等已验证趋势正在持续强化其竞争力。
- **探索构型** 应同步推进原型验证，但向规模部署转化需要以 OCS 控制面成熟度和软件栈承接能力为前提判据。
- **方案选择不脱离负载画像**。MoE、长上下文和多模态正在重排系统瓶颈的优先级，不存在脱离工作负载的静态最优架构。

若后续需要将帕累托坐标系进一步落到具体产品条目与实测口径，可参考仍处于筹备阶段的 [SPI](#)。

### 对政策制定者：国产超节点的现实路径

超节点能力建设对我国智算产业具有基础性的战略意义。系统能力边界外移速度的指数级放大效应意味着：越早建立系统级边界外移能力，越能避免在后续代际中被拉开难以逆转的差距。这一判断对算力基础设施规划、供应链韧性建设和标准体系构建都有直接的参考价值。

对国产超节点而言，更现实的路径不是试图在单条技术曲线上做线性追赶，而是在关键节点上建立系统级能力边界外移能力。[第五章](#)已按证据等级和时间窗把关键技术变量分为“已验证趋势 / 工程推断 / 方向判断”三档，并给出了对五类参考设计的再判断。基于该分析，产业路径可概括为：

- **近期（当前代际）**：优先构建可交付、可验证、可演进的标准构型——在当前能力边界上找到可站稳的位置。重点突破方向包括 Scale-Up 互联协议自主化、以太网超节点方案的工程落地和仿



真验证闭环的建立。

- **中期（2–3 年）**：围绕第五章标记为“工程推断”级别的变量（OCS 光交换、CPO、Chiplet + UCIe、HBM4）建立验证闭环，为下一代方案储备设计维度。
- **长期（3–5 年）**：把硬件平台、软件系统、仿真资产和参考设计方法论打通，形成持续迭代的技术路线——使能力边界外移成为一种可复用的系统能力，而不是一次性的产品拼装。

这一路径的核心逻辑是：超节点竞争不是单产品竞争，而是跨芯片、互联、光学、封装、整机、软件与运维的协同工程。建立这种协同能力需要产学研用各方的深度参与——芯片厂商提供算力底座、光模块厂商提供互联能力、封装厂商突破集成极限、云厂商验证规模化落地、高校和研究机构提供前沿理论和算法支撑。只有当这些角色在共同的分析框架下协同推进，系统能力边界外移才能从概念变为持续的工程实践。

为使这种协同有据可依，本白皮书配套提出了 [SuperPod Pareto Index \(SPI\)](#) 这一筹备中的说明书。SPI 当前并不被表述为已经定型的正式机制，而是把产品条目边界、证据等级、保密处理、治理分工、版本演进和争议处理等关键问题先摆到台面上，供产业界逐步共建。白皮书提供分析坐标系，SPI 则承担把后续协作接口提前说明清楚的作用，其价值正在于为产业界预留共同建设规则、证据和治理流程的空间。

## 对研究者：开放问题与未来方向

本白皮书建立的系统能力边界分析框架，并用帕累托前沿将其形式化，为超节点领域提供了一个统一的分析坐标系。但这个框架本身也面临若干有待后续研究回答的开放问题：

1. **异构算力场景下的帕累托前沿形状**：当系统中同时包含 GPU、NPU、FPGA 等异构加速器时，帕累托前沿的维度和形状如何变化？异构混合编排是否会引入新的不可支配关系？
2. **光交换引入后的动态帕累托曲面**：传统分析假设拓扑在任务执行期间保持静态。当 OCS 或可重构光交换使拓扑可以动态调整时，帕累托前沿是否应从静态曲线扩展为时变曲面？
3. **模型-硬件协同演化下的前沿预测**：MoE、长上下文、多模态等模型架构变化会改变通信模式和资源需求画像。是否可以建立模型架构变化对帕累托前沿形变的预测模型？
4. **能力边界外移的经济学分析**：每一次边界外移都有工程成本。在给定投资约束下，应优先在哪些维度上投入以获得最大的边界外移幅度？是否存在最优的维度选择策略？
5. **超节点的可组合性与模块化**：当 Chiplet、先进封装和可重构互联使系统组件更加模块化时，是否可以建立“可组合能力边界”的理论框架，使边界外移本身变得更加灵活和可复用？

这些问题的回答将进一步丰富帕累托前沿外推框架的解释力和预测力，也为超节点领域的持续研究提供了清晰的方向。

超节点竞争的胜负，最终取决于谁能以更快的速度、在更多的维度上持续推动系统能力边界外移。这也是本白皮书希望为产业、研究与决策三方共同提供的基本判断。

# SuperPod Pareto Index (SPI)

!!! info "筹备中" SPI 目前处于早期筹备阶段，尚未形成正式评估机制。以下内容仅为方向性说明，具体方案将在后续版本中逐步完善。

## 概述

SuperPod Pareto Index (SPI) 计划建立一套基于本白皮书多维分析框架的超节点产品整理与比较机制，帮助产业各方在统一的坐标系下讨论不同技术路线的系统级能力。

## 目标

- 将白皮书的多维分析框架延伸为可操作的产品评估入口
- 为不同技术路线的超节点方案提供结构化的比较基础
- 为企业、研究机构与编写委员会之间的协同预留接口

## 当前进展

当前仓库中已预留 SPI 基础结构（`spi/` 目录），包含产品模板和 schema 定义，可供试填和讨论。已收录的产品条目见 [产品入口](#)。

## 参与方式

欢迎通过 **GitHub Pull Request** 参与 SPI 的筹备工作，包括但不限于：产品条目试填、评估框架建议、证据口径讨论等。

